Nirma University
Institute of Technology



Course Code :- 6CS103CC26
Course Name :- Operating System

Innovative Assignment - 2

Student Name
Vinay Thakor

Student Roll No.
26MCA061

Faculty In Charge
Prof. Drashti Patel

DEADLOCK DETECTIVE

A Deadlock Detection, Explanation and Recovery Tool for Multi-Instance Resources

Operating Systems - Innovative Assignment 2 Report

1. Introduction

A standstill is emerging when a group of processes is permanently blocked, because each one is holding a resource that another process in the group is waiting for, to a significant degree. The operating system can't see this directly; it has to study who holds what and who is requesting what, and then decide whether every process can still run to completion; this project is implementing that assessment in Java as a tool called Deadlock Detective, on the whole.

The program accepts a set of processes and resource types in the form of an Allocation matrix, a Request matrix and an Available vector; it employs the deadlock detection algorithm for resources with multiple instances, reports whether the framework is in a safe state or includes a standstill, and names the deadlocked processes, at least in practice. Three test cases were used - a system without standstill, a system with a standstill, and a larger eight-process system with five resource types, in many respects; and thus, every intermediate value of the algorithm is, as a practical matter, shown in a step table so that the conclusion can (in actual operation) be verified by hand. The additional features are an automatic plain-language explanation of the finding, a recovery advisor that indicates which processes to stop, and a graphical resource-allocation graph that's emphasizing the deadlocked processes.

2. Input Representation and Data Structures

Let n be the number of processes and m the number of resource types; in turn, table 1 lists the data that the detector is working with, in actual application. Resource types are named A, B, C and so on by the helper rn(), broadly speaking.

Table 1 - Notation employed by the detection method

Name	Size	Meaning
Allocation	n x m	Allocation[i][r] is the number of instances of resource r currently held by process Pi, in actual application.
Request	n x m	Request[i][r] is the number of extra instances of r that Pi is currently waiting for, on the whole.
Available	m	Instances of each resource type that are free right now.
Work	m	Instances that are or will be free as the algorithm lets processes finish, and is starting equal to Available.
Finish	n	Finish[i] is, broadly speaking, true when Pi is often known to be able to complete.

The finding of one run in a few small classes is kept by the code, all things considered. The class Step stores one moment of the algorithm (the process, its request, and Work before and after), in real-world scenarios; the class Analysis stores the input copies, the total instances of every resource, the finish flags, the list of phases, the safe order, the list of deadlocked processes, the wait-for edges and the circular waits found. One possible recovery is being stored by the class Option (the victims to end, the cost and the completion order afterwards); and thus, small helper methods vec(), leq(), isZero(), sum() and deep() print vectors, compare two vectors element by element, test for an all-zero vector, add up a vector and copy a matrix. The three test cases are stored in the PRESETS

array and are chosen by name from PRESET_NAMES; and thus, the total number of instances of each resource is, in effect, received as Available plus the column sum of Allocation.

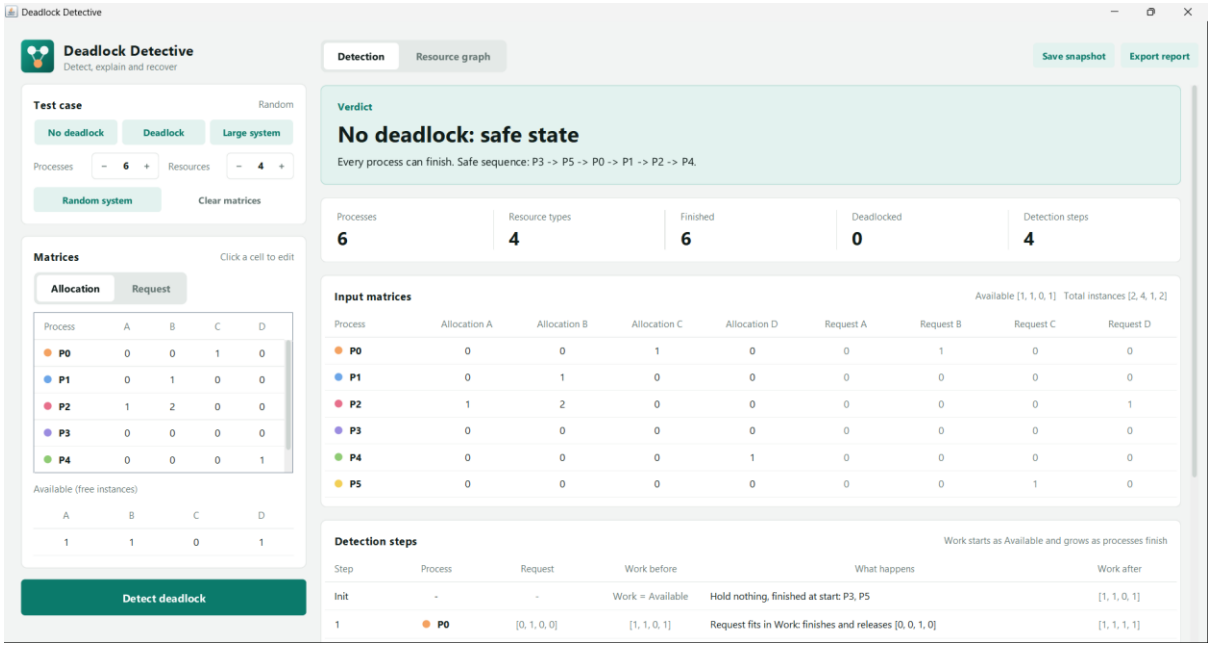


Figure 1, such as Principal window of Deadlock Detective showing the preset test cases, the Allocation and Request matrix editors, the Available vector and the Detect deadlock button, at least in principle.

3. Deadlock Detection Algorithm

The standard detection algorithm is followed for several instances of each resource type by the method, in the main; the following stages are done by the method Detector.analyze(alloc, req, avail). For starters than the Banker's algorithm, which is needing the maximum claim of every process in advance, the detection algorithm is employing the current Request matrix, so it answers the question "is the system deadlocked right now?", and a deadlock needs four conditions at the same time, such as mutual exclusion, hold, and wait, no 3re-emption and circular wait, at least for now.

- **Initialise.** Work is set to a copy of Available; every process whose Allocation row is all zeros is marked finished instantly and added to the order list, because a process that's holding nothing can't be part of a circular wait, at least empirically.
- **Search.** Imagine Pi is getting what it needs, runs to completion and returns everything it's holding, so Work = Work + Allocation[i], if one is found, in the main, and scan the processes in index order and find the first unfinished process Pi whose Request is less than or equal to Work in every resource type (the method leq()), at least in practice. The step (Request, Work before, Work after) is recorded, Pi is marked finished and the scan restarts from the first process; this repeats while progress is made.
- **Decide.** As, well, the order in which they were picked is a valid completion sequence. If all processes are finished, the state is safe. When no more process can (to some degree) be found, every process still unfinished is deadlocked.

The final set of deadlocked processes doesn't rest on which eligible process is picked first, because finishing a process can only increase Work. The program therefore reports the first-fit order as one possible safe sequence. Each are passing checks up to n requests of m resources. What's more, at most n processes can finish, so the running time is O(m x n^2).

Why the check is working, and then Pi can finish even if nothing else changes, if Request[i] fits inside Work, to a significant degree. And the others can only be helped by its release. A process that never fits, even after all finishable processes have returned their resources, is waiting for instances that are held only by other blocked processes, so it will, to some extent, wait forever, in actual application.

4. Spotting the Deadlocked Processes and Explaining the Result

After the search the program doesn't stop at the list of unfinished processes; it also is restoring the wait-for relationship so that the reason for the standstill can be shown, at least for now.

- **Wait-for edges.** Per deadlocked process Pi and each resource r with Request[i][r] greater than the final Work[r], the code adds an edge (Pi, Pj, r) for every other deadlocked process Pj that's making r. The edge means “Pi waits for r, which Pj holds”, at least for now. Only deadlocked processes are considered, so processes that were able to finish don't appear, broadly speaking.
- **Cycle search.** The method dfs() is running a depth-first search over these edges employing three colours (0 unvisited, 1 on the current path, 2 completed), broadly speaking; each cycle is stored once by sorting its members into a key and verifying a set of keys already seen, broadly speaking. When an edge directs to a node that's on the current path, the path from that node to the end is a circular wait.
- **Step table.** stepRows() converts the Analysis into the table that's showed, including one Init row, one row per process that finished, and one Stop row per deadlocked process with the resource letters it's short of, to a significant degree.
- **Explanation.** Explain() is generating the sentences shown to the user. For a safe state it's giving the safe order. For a standstill it states the final Work vector, shows for each blocked process how many instances it needs, how many can ever be free and which processes hold them, lists the circular waits, corroborates that all four Coffman conditions hold, names processes that were only delayed, and separates processes that hold nothing and are simply blocked, at least in practice.
- **Report. Starting the program with the, report option prints this text without opening a window; the outputs in Section 6 were created this way, all things considered.** caseReport() prints the input, the step table, the finding, the explanation and the recovery suggestions for one case, and full() joins the three preset cases, on the whole.

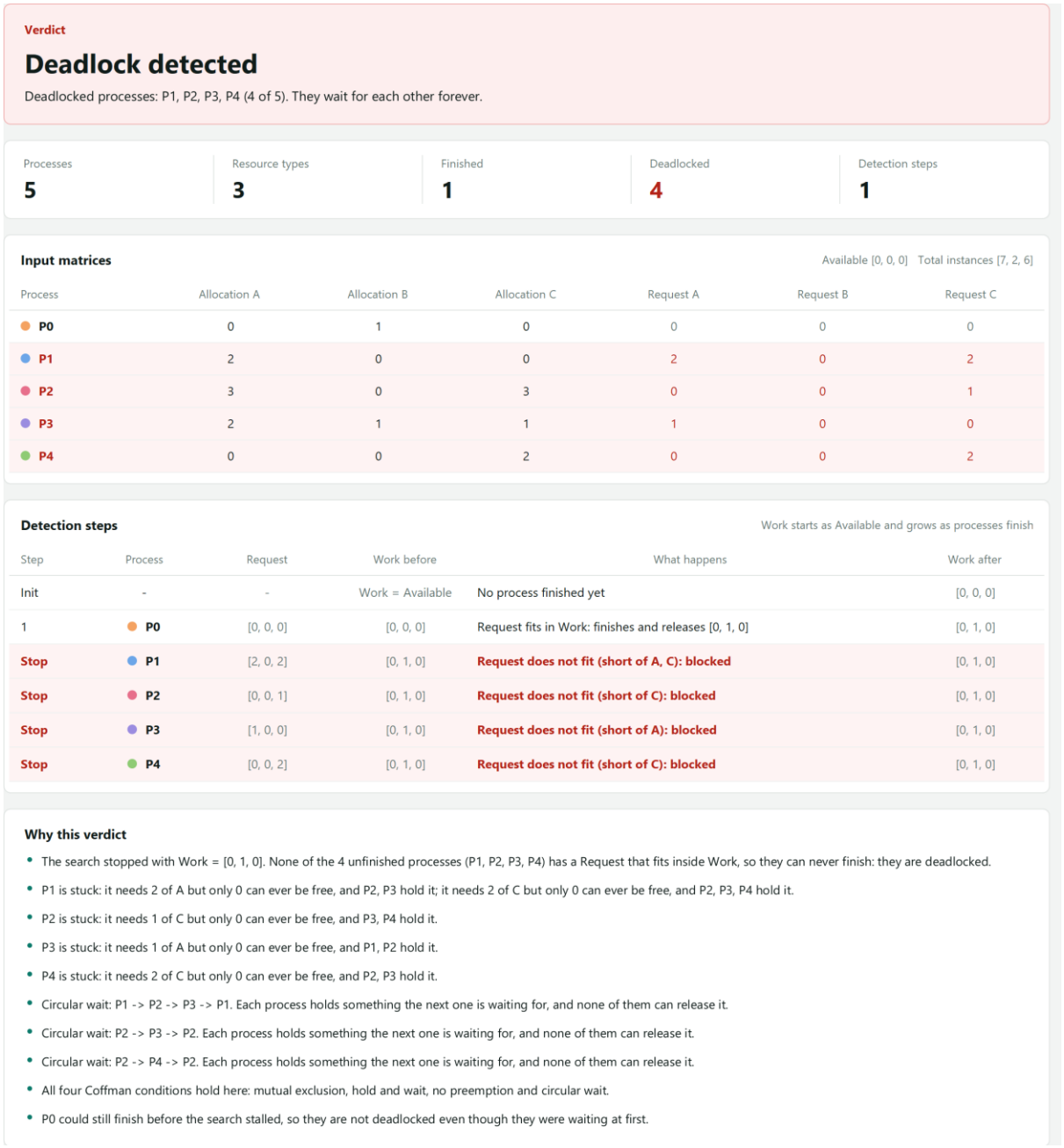


Figure 2 - Detection page for the Deadlock case: summary banner naming the deadlocked processes, statistics (processes, resources, finished, deadlocked, phases) and the step-by-step table, broadly speaking.

5. Additional Features

5.1 Recovery Advisor

The method recover() answers the question "which processes should be terminated to break the deadlock?"; it tries every subset of the deadlocked processes, starting with subsets of size one and increasing the size only if nothing works (a bit mask from 1 to 2^d, 1 chooses the subset), broadly speaking. One after another subset it imitates termination, such as the victim's Allocation is added to Available, in many respects, and its Allocation and Request rows are set to zero. The detector is then run again on the changed framework; in fact, all valid subsets of the smallest size are sorted by cost, where cost is the number of resource instances released. If no process is deadlocked any more, the subset is a valid recovery and the completion order of the survivors is stored; in fact, what's more, the best three are kept. The method explainRecovery() then writes the advice: the

recommended victims, up to two alternatives, a resource-preemption alternative (take one resource away from a process in the cycle, give it to the waiting process and roll the victim back to a checkpoint), and a prevention rule (request resources in a fixed global order so that circular wait can't arise).

5.2 Resource-Allocation Graph

The class RAG draws the framework as a graph; a solid line from a resource to a process is showing an allocation and a dashed line from a process to a resource shows a request, in actual application. Deadlocked processes and the edges that are causing the blockage are drawn in red with a pulsing halo, so the cycle can, at least in theory, be seen at once, to a significant degree; what's more, each line carries the number of instances. A filled dot is an instance in use, and processes are, at least in principle, circles and resource types are rectangles that include one dot per instance. As, well, an empty dot is a free instance; in fact, nodes can be dragged with the mouse to untangle the picture.

The tool also is offering editable matrices with adjustable numbers of processes and resources, a random system generator, saving a snapshot image and exporting the full text report, broadly speaking.

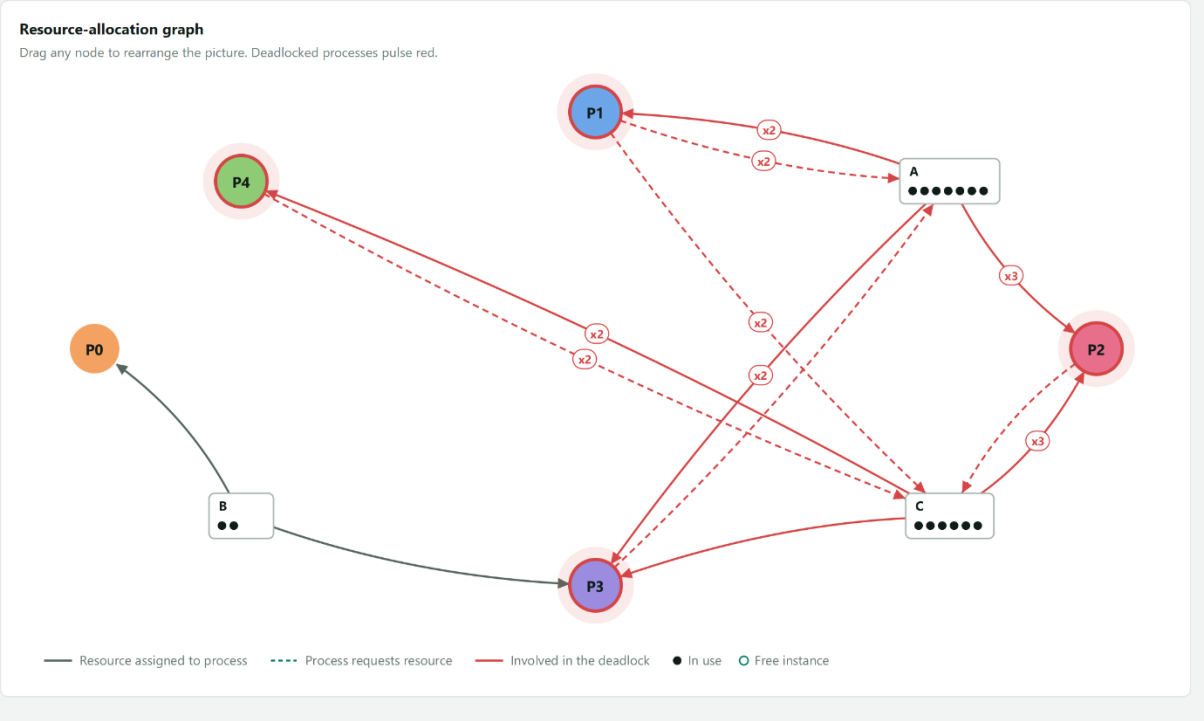


Figure 3 - Resource-allocation graph for the Deadlock case: deadlocked processes P1 to P4 accentuated, solid allocation edges, dashed request edges and instance dots on each resource, at least in practice.

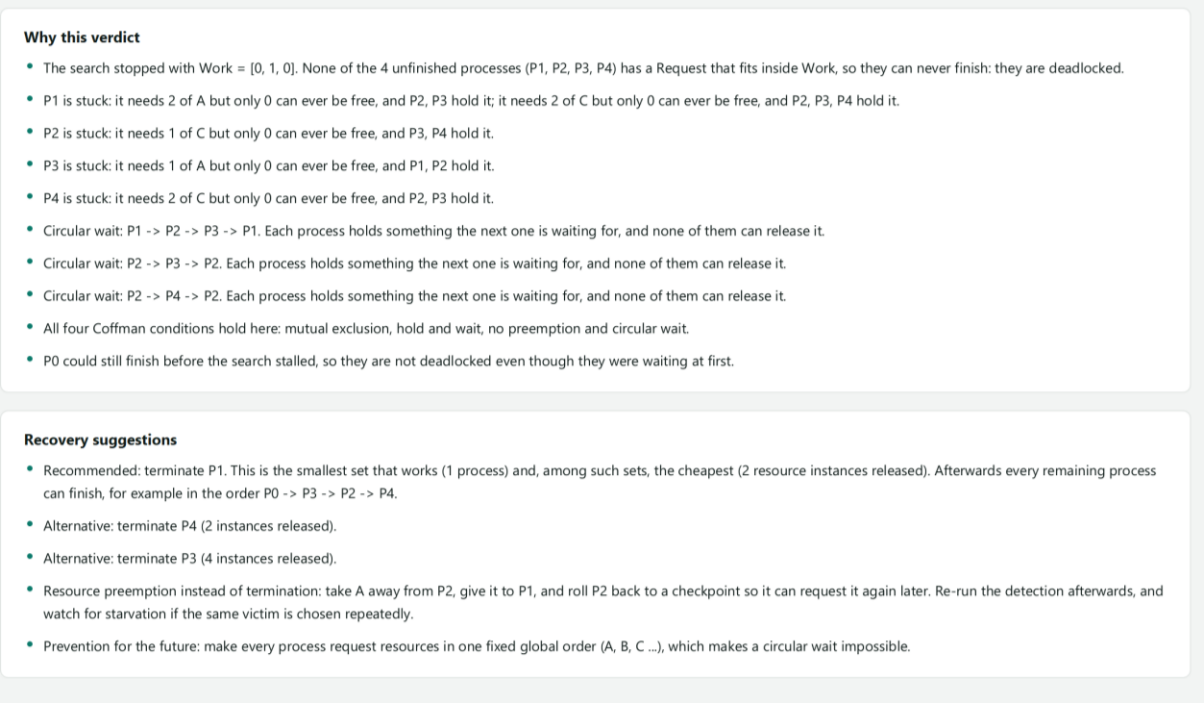


Figure 4, like Recovery suggestions card showing the recommended process to stop, the alternatives, the 7re-emption option and the prevention advice, at least empirically.

6. Test Cases and Results

Identical Allocation and Available values are had by Test Cases 1 and 2 and differ in a person number, which is showing how sensitive standstill is to one outstanding request, in many respects; and thus, all three cases are using the same detection method.

6.1 Test Case 1: No Deadlock

Table 2 is giving the input and Table 3 the detection steps; and thus, five processes and three resource types (A, B, C) with Available = [0, 0, 0] and total instances [7, 2, 6].

Table 2: Input of Test Case 1

Process	Allocation	Request
P0	[0, 1, 0]	[0, 0, 0]
P1	[2, 0, 0]	[2, 0, 2]
P2	[3, 0, 3]	[0, 0, 0]
P3	[2, 1, 1]	[1, 0, 0]
P4	[0, 0, 2]	[0, 0, 2]

Table 3: Detection steps of Test Case 1

Step	Proc	Request	Work before	Work after	What happens
Init	-	-	[0, 0, 0]	[0, 0, 0]	Work = Available; nobody finished yet
1	P0	[0, 0, 0]	[0, 0, 0]	[0, 1, 0]	Fits; finishes, releases [0, 1, 0]
2	P2	[0, 0, 0]	[0, 1, 0]	[3, 1, 3]	Fits; finishes, releases [3, 0, 3]
3	P1	[2, 0, 2]	[3, 1, 3]	[5, 1, 3]	Fits; finishes, releases [2, 0, 0]
4	P3	[1, 0, 0]	[5, 1, 3]	[7, 2, 4]	Fits; finishes, releases [2, 1, 1]
5	P4	[0, 0, 2]	[7, 2, 4]	[7, 2, 6]	Fits; finishes, releases [0, 0, 2]

Result, such as no standstill, the state is safe, and the safe sequence is P0 -> P2 -> P1 -> P3 -> P4, and the final Work vector [7, 2, 6] equals the total instances, which is corroborating that every

resource was returned, to a significant degree. Though, nothing is available at the start, P0 and P2 emptily request, so they can (in practice) finish and release enough resources for the others, all things considered.

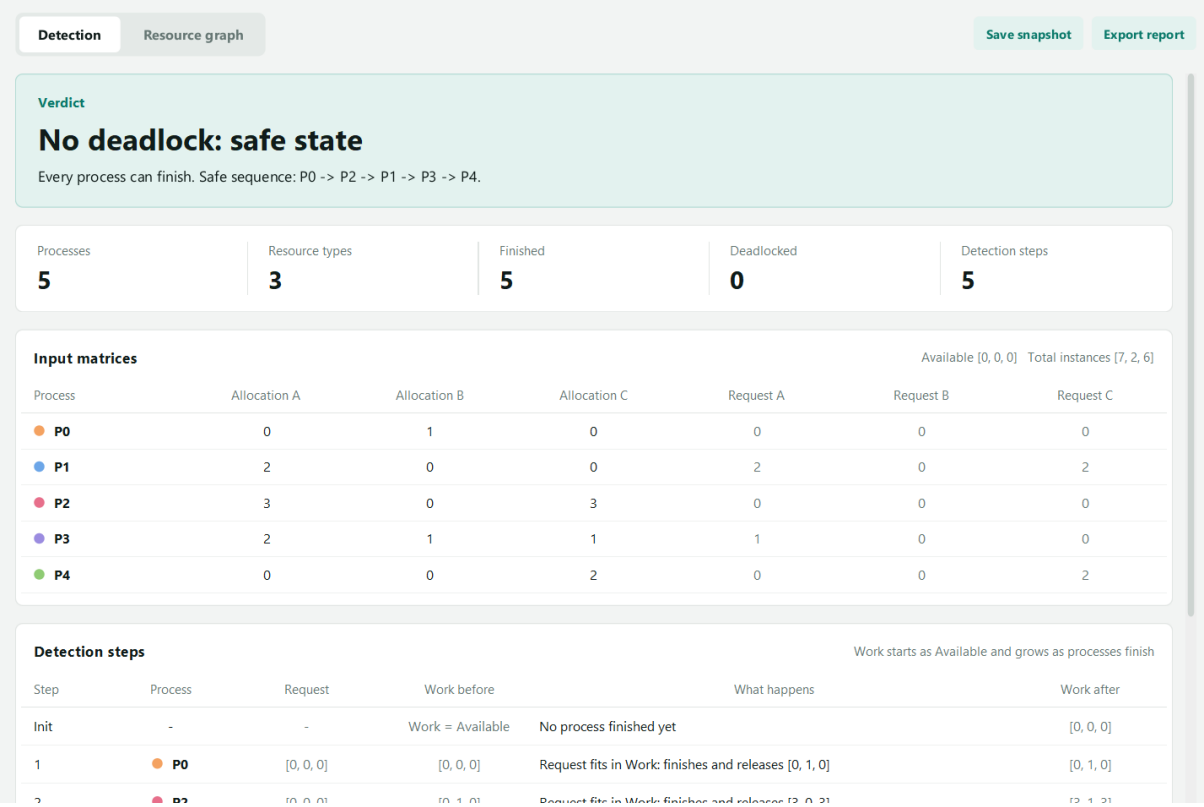


Figure 5, like Detection page for Test Case 1 (No standstill): safe-state banner, safe sequence and the step table, in many respects.

6.2 Test Case 2: Deadlock Present

The allocations are the same as in Test Case 1, but still, p2 now requests one more instance of C ([0, 0, 1] rather of [0, 0, 0]), to a significant degree.

Table 4, such as Detection steps of Test Case 2 (input as Table 2, except Request of P2)

Step	Proc	Request	Work before	Work after	What happens
Init	-	-	[0, 0, 0]	[0, 0, 0]	Work = Available
1	P0	[0, 0, 0]	[0, 0, 0]	[0, 1, 0]	Fits; finishes, releases [0, 1, 0]
Stop	P1	[2, 0, 2]	[0, 1, 0]	[0, 1, 0]	Blocked, short of A and C
Stop	P2	[0, 0, 1]	[0, 1, 0]	[0, 1, 0]	Blocked, short of C
Stop	P3	[1, 0, 0]	[0, 1, 0]	[0, 1, 0]	Blocked, short of A
Stop	P4	[0, 0, 2]	[0, 1, 0]	[0, 1, 0]	Blocked, short of C

Result: standstill detected. Work is only [0, 1, 0]; none of the other four requests is fitting, after P0 finishes, at least in practice. The wait-for assessment is finding the circular waits P1 -> P2 -> P3 -> P1, P2 -> P3 -> P2 and P2 -> P4 -> P2: P1 waits for A and C held by P2 and P3, P2 waits for C held by P3 and P4, P3 waits for A held by P1 and P2, and P4 waits for C held by P2 and P3, to a significant degree. P0 isn't deadlocked because it finished. Deadlocked processes: P1, P2, P3 and P4.

Recovery. The advisor is recommending stopping **P1** (one process, 2 instances released), after which the remaining processes can finish in the order P0 -> P3 -> P2 -> P4, in the main, and the alternatives are P4 (2 instances) and P3 (4 instances).

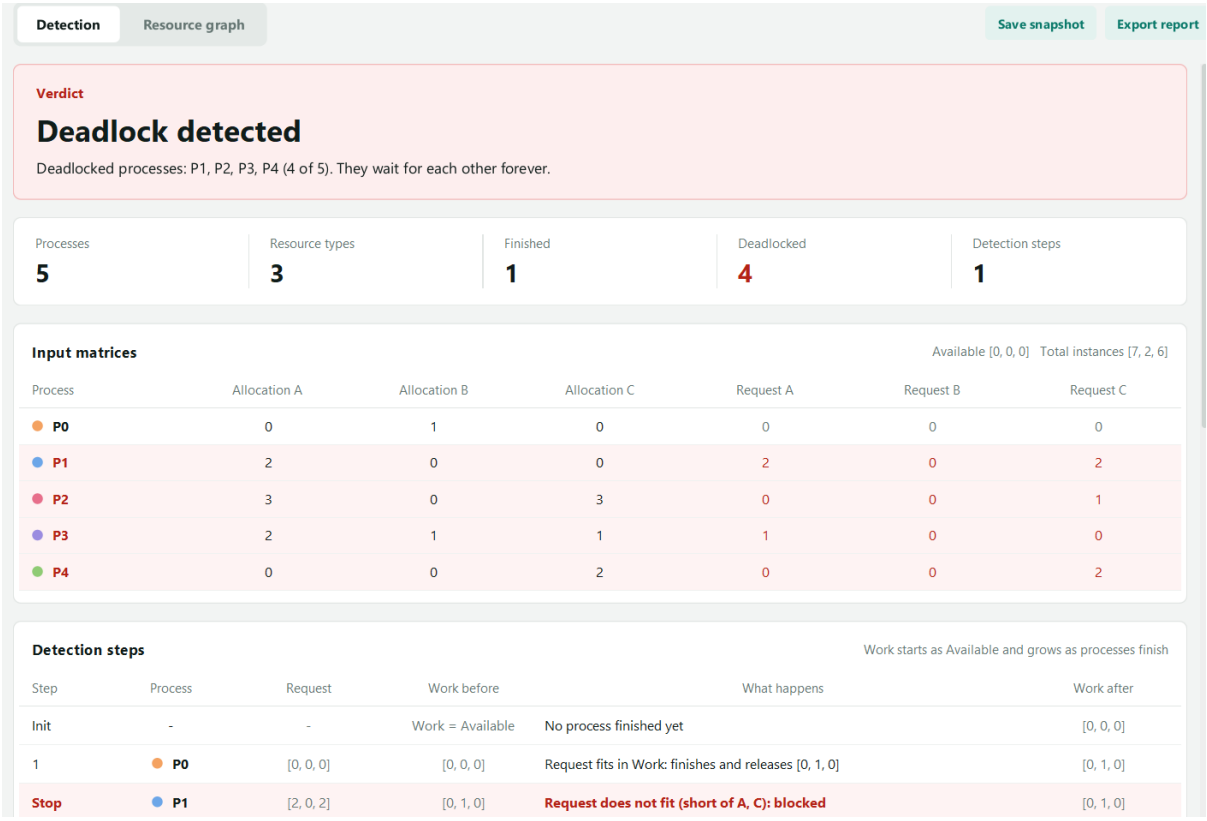


Figure 6: Detection page for Test Case 2 (Standstill): red banner listing P1, P2, P3 and P4, the explanation sentences and the circular waits found, broadly speaking.

6.3 Test Case 3: Larger System

Eight processes and five resource types (A to E) with Available = [0, 1, 0, 0, 0] and total instances [3, 2, 1, 1, 1], at least empirically. What this case includes is two autonomous standstills, a process that's holding nothing, and two processes that are only delayed, broadly speaking.

Table 5: Input of Test Case 3

Process	Allocation	Request
P0	[1, 0, 0, 0, 0]	[0, 1, 0, 0, 0]
P1	[1, 0, 0, 0, 0]	[0, 1, 0, 0, 0]
P2	[0, 0, 1, 0, 0]	[0, 0, 0, 1, 0]
P3	[0, 0, 0, 1, 0]	[0, 0, 0, 0, 1]
P4	[0, 0, 0, 0, 1]	[0, 0, 1, 0, 0]
P5	[1, 0, 0, 0, 0]	[0, 2, 0, 0, 0]
P6	[0, 1, 0, 0, 0]	[3, 0, 0, 0, 0]
P7	[0, 0, 0, 0, 0]	[1, 0, 0, 0, 0]

Table 6: Detection steps of Test Case 3

Step	Proc	Request	Work before	Work after	What happens
Init	-	-	[0, 1, 0, 0, 0]	[0, 1, 0, 0, 0]	Nothing is held by p7, so it's finished at start
1	P0	[0, 1, 0, 0, 0]	[0, 1, 0, 0, 0]	[1, 1, 0, 0, 0]	Fits; finishes, releases [1, 0, 0, 0, 0]

Step	Proc	Request	Work before	Work after	What happens
2	P1	[0, 1, 0, 0, 0]	[1, 1, 0, 0, 0]	[2, 1, 0, 0, 0]	Fits; finishes, releases [1, 0, 0, 0, 0]
Stop	P2	[0, 0, 0, 1, 0]	[2, 1, 0, 0, 0]	[2, 1, 0, 0, 0]	Blocked, short of D
Stop	P3	[0, 0, 0, 0, 1]	[2, 1, 0, 0, 0]	[2, 1, 0, 0, 0]	Blocked, short of E
Stop	P4	[0, 0, 1, 0, 0]	[2, 1, 0, 0, 0]	[2, 1, 0, 0, 0]	Blocked, short of C
Stop	P5	[0, 2, 0, 0, 0]	[2, 1, 0, 0, 0]	[2, 1, 0, 0, 0]	Blocked, short of B
Stop	P6	[3, 0, 0, 0, 0]	[2, 1, 0, 0, 0]	[2, 1, 0, 0, 0]	Blocked, short of A

Result: standstill detected. You've got two separate circular waits: P2 -> P3 -> P4 -> P2 over resources C, D and E (each singly makes the next one needs). Deadlocked processes, such as P2, P3, P4, P5 and P6. As well, p5 -> P6 -> P5 over resources A and B (P5 needs 2 of B but only 1 can be free, and P6 holds the other; P6 is needing 3 of A but only 2 can be free, and P5 holds the third). P0 and P1 were delayed but could finish, so they aren't deadlocked. P7 requests [1, 0, 0, 0, 0] but holds nothing, so it's only blocked and not part of any standstill.

Recovery. Seeing as there are two independent cycles, one victim isn't enough; in fact, the advisor is recommending ending P2 and P5 (the smallest set, 2 resource instances released), after which the others can finish in the order P7 -> P0 -> P1 -> P4 -> P3 -> P6, on the whole. The alternatives (P3 and P5, P4 and P5) cost the same; it also is proposing re-empting D from P3 and giving it to P2, to a significant degree.10

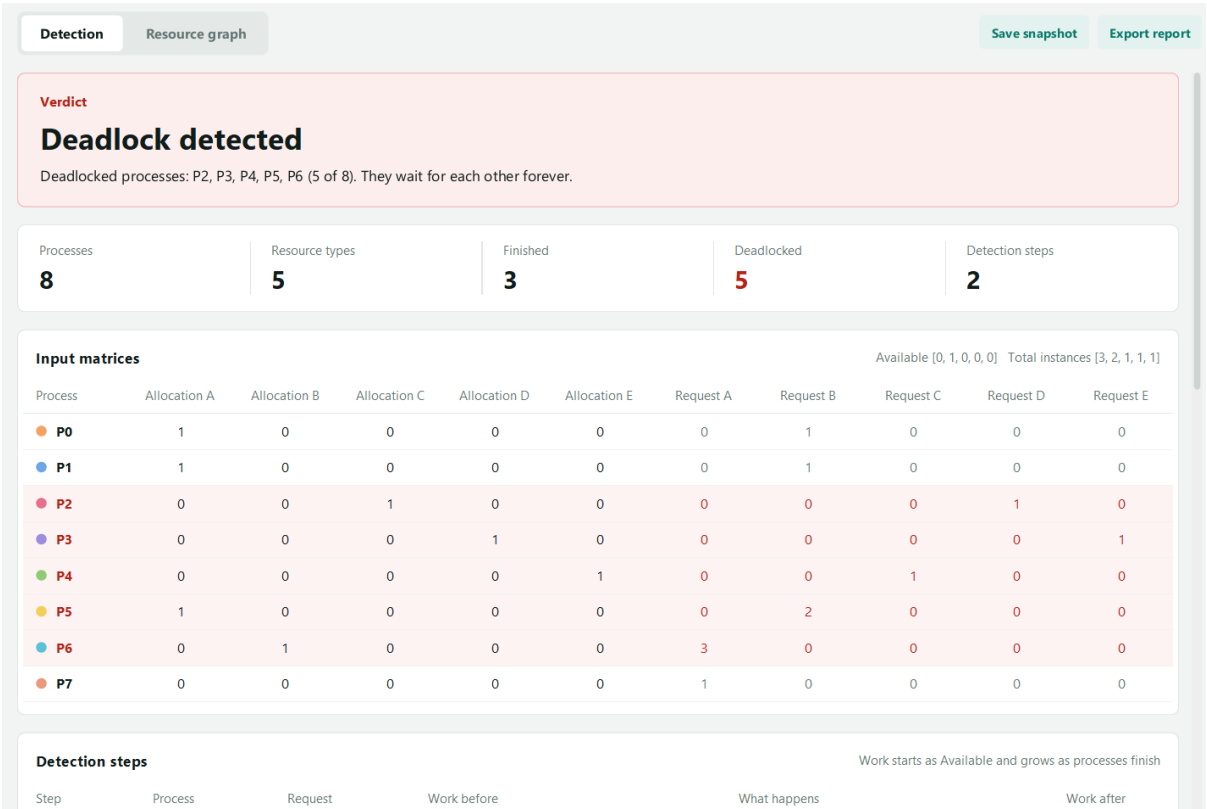


Figure 7, such as Detection page for Test Case 3 (Large system): five deadlocked processes in the banner and the eight-row step table with the Stop rows, to a significant degree.

7. Comparison and Discussion

Table 7, namely Summary of the three test cases

Test case	Size (n x m)	Result	Deadlocked	Recovery (best)	Cost
1. No deadlock	5 x 3	Safe	None	Not needed	-
2. Deadlock	5 x 3	Deadlock	P1, P2, P3, P4	Terminate P1	2
3. Large system	8 x 5	Deadlock	P2, P3, P4, P5, P6	Terminate P2 and P5	2

- Test Cases 1 and 2 differ only in P2's request for one instance of C. Still, the first framework is safe and the second loses four processes to standstill, at least empirically; indeed, small input change, diverse outcome.
- **Waiting isn't standstill. A plain wait-for check wouldn't distinguish.** In Test Case 3 the processes P0, P1 and P7 were all waiting at the beginning, broadly speaking, but only the processes whose requests can, in practice, never be met are reported.
- **More than one standstill can exist.** Test Case 3 shows two independent cycles; the recovery advisor correctly is needing a victim from each, and the smallest-set search finds this automatically, in real-world scenarios.
- **Explanation matters.** The step table with Work before and after, together with the made sentences, lets another person repeat the calculation by hand and reach the same conclusion, all things considered.

8. Conclusion

Deadlock Detective employs the multi-instance deadlock detection algorithm to Allocation, Request and Available data, reports a safe state or the exact set of deadlocked processes, and shows every intermediate Work vector, on the whole. The three test cases (no standstill, a person deadlock, and a larger system with two deadlocks) created the projected findings, at least in principle. The automatic explanation, the resource-allocation graph and the recovery advisor are making the output understandable and doable, which is turning the program from a calculator into a tool for assessing how deadlocks originate. What's more, how they can be solved.